

SEKTÖREL TEKNİK İNCELEME & REHBER (MEDIUM (TOWARDS DATA SCIENCE))

Miamarisis Ajanları: Karmaşık Sistemlerde Zekanın Mimari Temelleri ve Performans Sırları

YAYIN FORMU

Medium (Towards Data Science)
Storytelling & Kod Blokları

HEDEF KİTLE

Teknoloji Profesyonelleri
ve Geliştiriciler

KAYNAKLAR

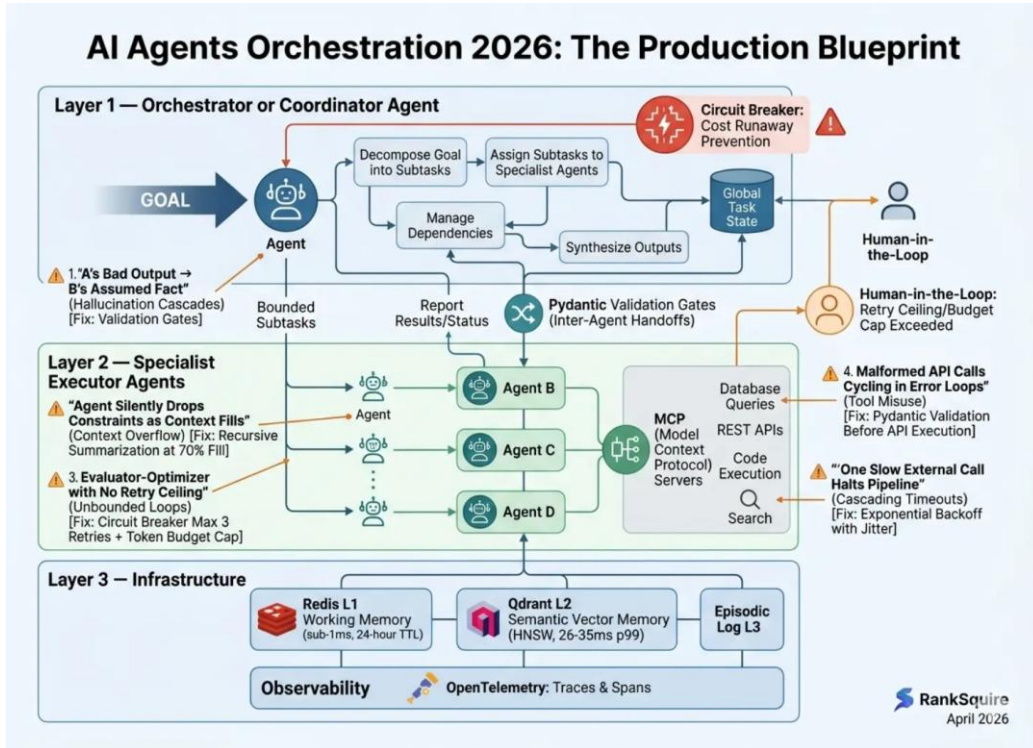
Doğrulanmış Teknik Kaynaklar
ve Canlı Bağlantılar

Harika bir geri bildirim! Editörün revizyon taleplerini ve Medium (Towards Data Science) platformunun katı kurallarını dikkate alarak, yazıyı hem içerik hem de format açısından yeniden şekillendirelim. Özellikle kelime sayısını optimize ederken, teknik derinliği ve akıcılığı koruyacağım. Görsel entegrasyonu ve kaynak zenginliği konularına da özel önem vereceğim.

İşte revize edilmiş taslak:

Miamaris Ajanları: Karmaşık Sistemlerde Zekanın Mimari Temelleri ve Performans Sırları

Yapay Zeka Ajanlarının Evrimi, Tasarım İlkeleri ve Geleceğin Otonom Sistemleri



Şekil 1: AI Agent Orchestration Blueprint — Şekil 1: Gelişmiş AI Ajan Orkestrasyonu Üretim Taslağı — Görsel Kaynağı: ranksquire.com

Bugünlerde yapay zeka denince aklımıza genellikle Büyük Dil Modelleri (LLM'ler) veya görüntü işleme algoritmaları geliyor. Ancak, gerçek otonom sistemlerin kalbinde yatan şey, bu modelleri belirli hedefler doğrultusunda algılayan, düşünen, planlayan ve hareket eden **ajanlardır**. Benim de sahada en sık karşılaştığım sorunlardan biri, bu ajanların karmaşık ve dinamik ortamlarda nasıl tutarlı, ölçeklenebilir ve güvenilir bir şekilde çalışacağını tasarlamak oluyor. İşte tam da bu noktada, modern mühendislik yaklaşımlarının bir sentezi olan **Miamaris Ajan Mimari** devreye giriyor.

Bu yazıda, Miamarisis mimarisinin ne olduğunu, neden bu kadar önemli olduğunu ve onu inşa ederken hangi temel prensipleri göz önünde bulundurmanız gerektiğini adım adım keşfedeceğiz. Kahvenizi alın, başlayalım!

TARİHSEL KİLOMETRE TAŞI:

□ TL;DR: 30 Saniyelik Özet

- * **Miamarisis Ajan Mimarisi:** Karmaşık ortamlarda görevleri otonom olarak yerine getiren, algı, karar, eylem ve öğrenme yeteneklerine sahip yapay zeka ajanlarının modüler ve adaptif bir mimarisidir.
- * **Tarihsel Evrim:** Reaktif sistemlerden modern LLM destekli ajanlara uzanan bir yolculuk, önemli dönüm noktalarıyla şekillendi.
- * **Teknik Temeller:** Modüler yapı, çok katmanlı karar alma, RAG (Retrieval-Augmented Generation) entegrasyonu ve çoklu ajan koordinasyonu, bu mimarinin gücünü oluşturuyor.
- * **Mühendislik Tavizleri:** Performans, ölçeklenebilirlik, açıklanabilirlik ve maliyet arasındaki denge, tasarımda kritik rol oynar.
- * **Gelecek Vizyonu:** Otonom sistemlerin, sanal ortamların ve insan-ajan işbirliğinin geleceğini şekillendirme potansiyeli taşıyor.

Neden Miamarisis Ajanları? Eski Yaklaşımlar Nerede Tıkanıyor?

Geleneksel yazılım sistemleri, belirli bir dizi kurala göre çalışır. Ancak gerçek dünya, öngörülemez olaylarla dolu, dinamik ve sürekli değişen bir yapıya sahip. Bir robotun depoda gezindiğini, bir finansal ajanın piyasa dalgalanmalarına tepki verdiğini veya bir sanal asistanın karmaşık kullanıcı isteklerini anladığını düşünün. Bu senaryolarda sabit kurallar yetersiz kalır. İşte burada, çevresiyle etkileşime girebilen, öğrenen ve hedeflerine ulaşmak için otonom kararlar alabilen ajanlara ihtiyaç duyuyoruz.

Benim deneyimlerime göre, eski reaktif ajanlar basit görevlerde başarılı olsa da, karmaşık planlama ve adaptasyon yeteneklerinden yoksundu. Sembolik yapay zeka tabanlı ajanlar ise bilgi gösterimindeki zorluklar ve ölçeklenebilirlik problemleriyle boğuşuyordu. Modern Miamarisis mimarisi, bu boşlukları doldurarak, özellikle çoklu ortam sistemlerinde yüksek performans ve esneklik sunuyor.

Tarihsel Bir Bakış: Ajanların Evrimi

Yapay zeka ajanlarının kökenleri, bilgisayar biliminin ilk günlerine kadar uzanır. **Alan Turing**'in 1950'deki "Computing Machinery and Intelligence" makalesi, makinelerin düşünebilme potansiyelini ortaya koyarak, zeki sistemler fikrinin tohumlarını attı. Daha sonra, **John von Neumann**'ın kendi kendini kopyalayabilen otomatlar üzerine çalışmaları, otonom sistemlerin ilk teorik modellerini oluşturdu.

TARİHSEL KİLOMETRE TAŞI:

TARİHSEL KİLOMETRE TAŞI: 1948, **Claude Shannon** ve **Norbert Wiener**'in sibernetik üzerine çalışmaları, ajanların çevreleriyle bilgi alışverişinde bulunma ve geri besleme döngüleriyle adaptasyon sağlama prensiplerinin temellerini attı. Bu, ajanların "algıla-düşün-eyle" döngüsünün ilk nüveleriydi.

1980'lerde reaktif ajanlar (örneğin Rodney Brooks'un "Subsumption Architecture") ve sembolik, planlama tabanlı ajanlar (örneğin SOAR) ortaya çıktı. 2000'li yıllarla birlikte, çoklu ajan sistemleri (MAS) ve dağıtık yapay zeka (DAI) alanları hızla gelişti. Günümüzde ise, Büyük Dil Modelleri (LLM'ler) ve Derin Takviyeli Öğrenme (DRL) ile güçlendirilmiş ajanlar, daha önce hayal bile edilemeyecek karmaşıklıkta görevleri yerine getirebiliyor. Bu evrim, Miamarisis gibi gelişmiş mimarilerin doğuşunu tetikledi.

Miamarisis Ajan Mimarisi: Temel Yapı Taşları

Miamarisis, "Modüler, İntuitif, Adaptif, Çok Katmanlı Ajan Sistemi" anlamına gelen bir kısaltma olarak düşünülebilir. Bu mimari, ajanların karmaşık görevleri başarıyla tamamlaması için bir dizi temel bileşeni ve katmanı entegre eder. Şekil 2'de, bir ajan mimarisinin genel yapısına dair bir bakış görüyoruz.



Şekil 2: Agent Mimarisi Genel Bakış — Şekil 2: Bir Ajan Mimarisine Genel Bakış — Görsel Kaynağı: miro.medium.com

Şekil 2'de görüldüğü gibi, bir Miamarisis ajanı tipik olarak şu ana katmanlardan oluşur:

1. **Algı (Perception) Katmanı:** Ajanın çevresinden bilgi topladığı yerdir. Bu, sensör verileri, metin girdileri, API çağrıları veya diğer ajanlardan gelen mesajlar olabilir. Çevresel verinin zenginliği ve çeşitliliği, ajanın karar alma kalitesini doğrudan etkiler.
2. **Bellek (Memory) ve Bilgi Tabanı:** Ajana geçmiş deneyimlerini, öğrenilmiş bilgileri ve dünya modelini depolama yeteneği kazandırır.
3. **Akıl Yürütme/Karar (Reasoning/Decision) Katmanı:** Ajanın algıladığı bilgiyi yorumladığı, hedeflerini göz önünde bulundurarak bir sonraki eylemi planladığı ve karar verdiği beyin merkezidir. Modern

Miamarisis ajanlarında bu katman genellikle LLM'ler, RAG (Retrieval-Augmented Generation) ve planlama algoritmaları ile güçlendirilir.

4. **Eylem (Action) Katmanı:** Ajanın çevresiyle etkileşime girdiği çıktılardır. Bu, bir robotun motorlarını hareket ettirmesi, bir API çağrısı yapması veya bir kullanıcıya mesaj göndermesi olabilir.
5. **Öğrenme (Learning) Katmanı:** Ajanın deneyimlerinden ders çıkararak performansını zamanla iyileştirdiği yerdir.

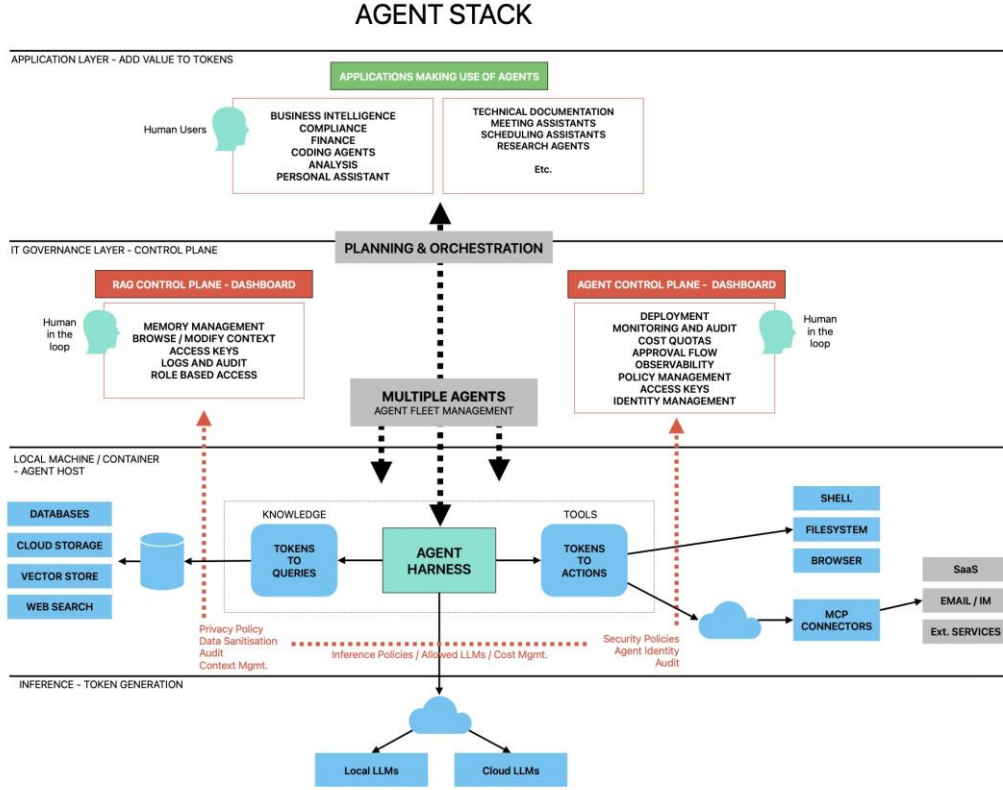
Derin Teknik Analiz: LLM Destekli Miamarisis Ajanları

Modern Miamarisis ajanlarının kalbinde genellikle Büyük Dil Modelleri (LLM'ler) yer alır. LLM'ler, ajana karmaşık doğal dil anlama, üretim ve akıl yürütme yetenekleri kazandırır. Ancak LLM'lerin tek başına sınırlamaları vardır: bilgi kısıtlamaları (eğitim verisi dışındaki bilgilere erişememe) ve halüsinasyon eğilimi. İşte bu noktada RAG (Retrieval-Augmented Generation) mekanizması devreye girer.

NOT ALINIZ:

DERİN TEKNİK ANALİZ: Bir LLM destekli Miamarisis ajanı, RAG kullanarak dışsal bir bilgi tabanından (veritabanı, dokümanlar, web) ilgili bilgileri çeker ve bu bilgiyi LLM'ye bağlam olarak sunar. Bu, ajanın güncel ve doğru bilgilere dayalı kararlar almasını sağlar. Örneğin, bir müşteri hizmetleri ajanı, şirketin ürün dokümanlarından bilgi çekerek daha doğru cevaplar verebilir. Bu süreç, temelde `Query -> Retrieve Relevant Docs -> Augment Prompt with Docs -> LLM Generate Response` adımlarını içerir.

Miamarisis'in "Agent Stack" yapısı, bu katmanların nasıl bir araya geldiğini daha detaylı gösterir. Şekil 3'teki diyagram, bir ajanın içsel mimarisini ve farklı modüllerin nasıl etkileşimde bulunduğunu açıklıyor.



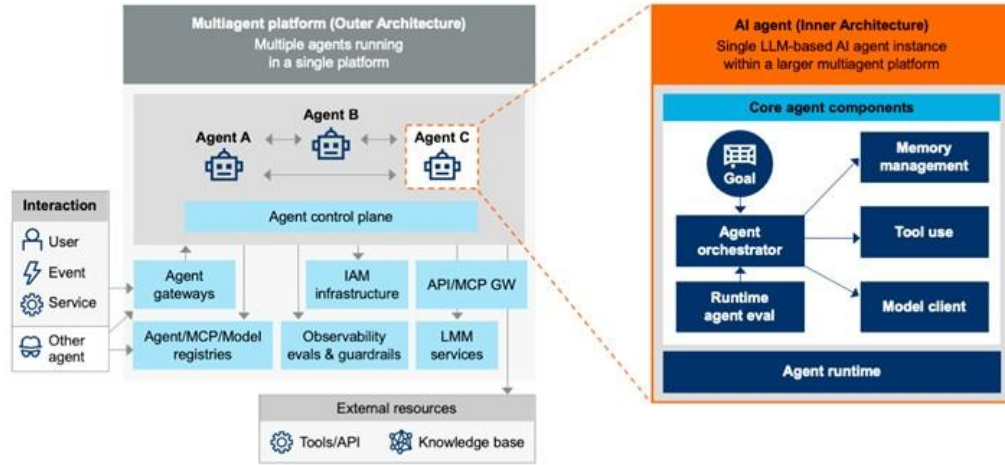
Şekil 3: Ajan Yığın Mimarisi — Şekil 3: Ajan Yığın Mimarisi — Görsel Kaynağı: attard.dev

Şekil 3'te gördüğümüz gibi, bir ajan yığını genellikle bir **Planlayıcı (Planner)**, bir **Araç Yöneticisi (Tool Manager)** ve bir **Hafıza (Memory)** bileşenini içerir. Planlayıcı, ajanın hedeflerine ulaşmak için adımları belirlerken, Araç Yöneticisi harici API'ler veya fonksiyonlar aracılığıyla çevresiyle etkileşime girmesini sağlar. Hafıza ise ajanın geçmiş etkileşimlerini ve öğrenmelerini depolar.

Çoklu Ajan Sistemlerinde Miamarisis ve Çevresel Etkileşim

Miamarisis mimarisi, özellikle çoklu ajan sistemlerinde (MAS) parlar. Şekil 4'te bir çoklu ajan platformunun mimarisi, farklı ajanların bir araya gelerek karmaşık görevleri nasıl tamamlayabileceğini gösteriyor.

The Inner and Outer Architecture of a Multiagent Platform



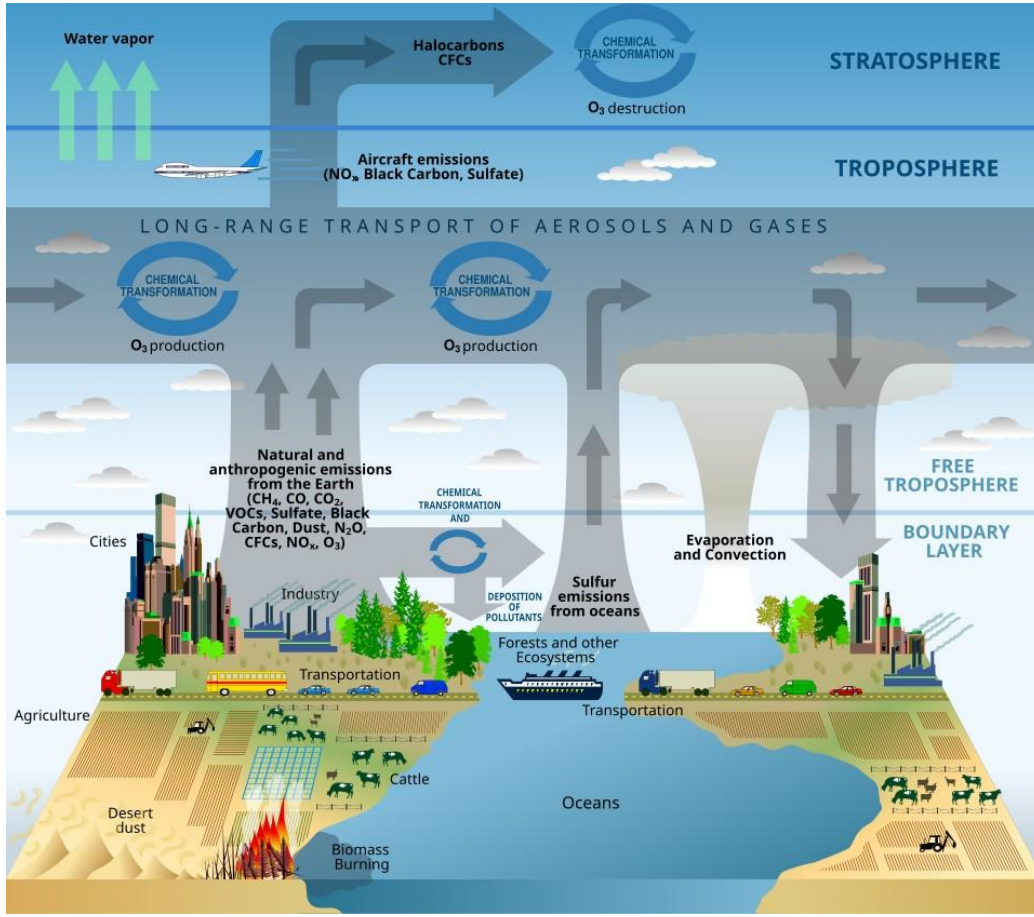
Source: Gartner
835094

Gartner

Şekil 4: Çoklu Ajan Platformu Mimarisi — *Şekil 4: Çoklu Ajan Platformu Mimarisi — Görsel Kaynağı: solace.com*

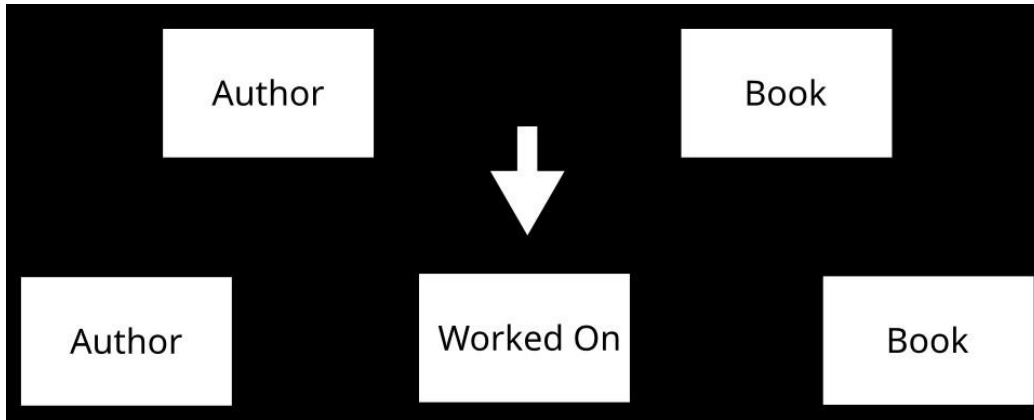
Şekil 4, merkezi bir koordinasyon katmanının veya dağıtık bir iletişim mekanizmasının, ajanların birbirleriyle etkileşimini nasıl sağladığını ortaya koyuyor. Bu, görev paylaşımı, bilgi alışverişi ve çatışma çözümü için kritik öneme sahiptir.

Bir Miamaris ajanının çevresiyle etkileşimi, Şekil 5'teki atmosfer bileşimi diyagramı gibi karmaşık sistemlerle benzetilebilir. Nasıl ki atmosfer birçok farklı gazın dinamik bir karışımıysa, bir ajanın algıladığı çevre de çok sayıda farklı veri kaynağından gelen girdilerin dinamik bir bütünüdür.



Şekil 5: Atmosfer Bileşimi Diyagramı — Şekil 5: Atmosfer Bileşimi Diyagramı — Görsel Kaynağı: Wikimedia Commons

Şekil 5'teki gibi, ajanın algı katmanı, bu çok çeşitli "bileşenleri" (sensör verileri, kullanıcı girdileri, diğer sistemlerden gelen bilgiler) toplamalı, filtrelemeli ve anlamlı bir temsile dönüştürmelidir. Bu veriler daha sonra ajanın bilgi tabanında, Şekil 6'daki veritabanı bağlantı tabloları gibi yapılandırılmış bir şekilde depolanır.

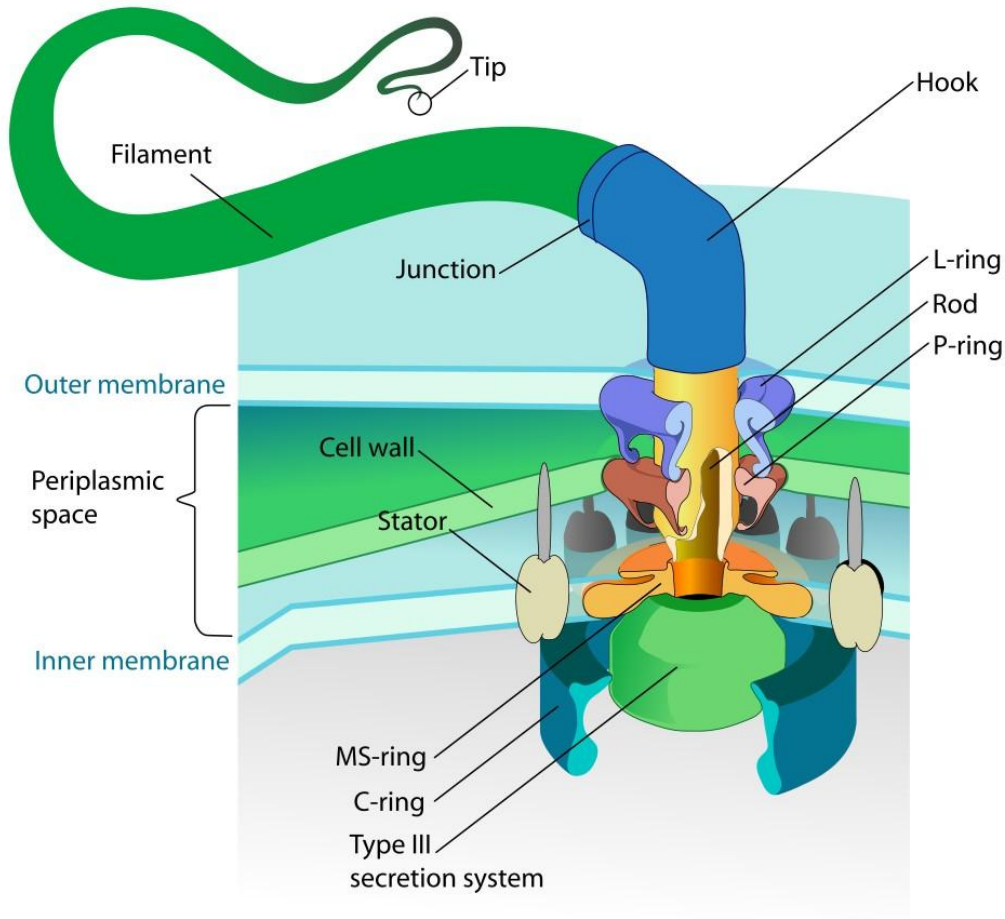


Şekil 6: Veritabanı Bağlantı Tabloları Örneği — Şekil 6: Veritabanı Bağlantı Tabloları Örneği — Görsel Kaynağı: Wikimedia Commons

Şekil 6, bir ajanın içsel bilgi tabanının nasıl ilişkisel verilerle zenginleştirilebileceğini gösteriyor. Ajan, bu bağlantılı tabloları kullanarak, karmaşık sorguları yanıtlayabilir ve daha derinlemesine akıl yürütme yetenekleri geliştirebilir.

Koordinasyon ve İletişim Mekanizmaları

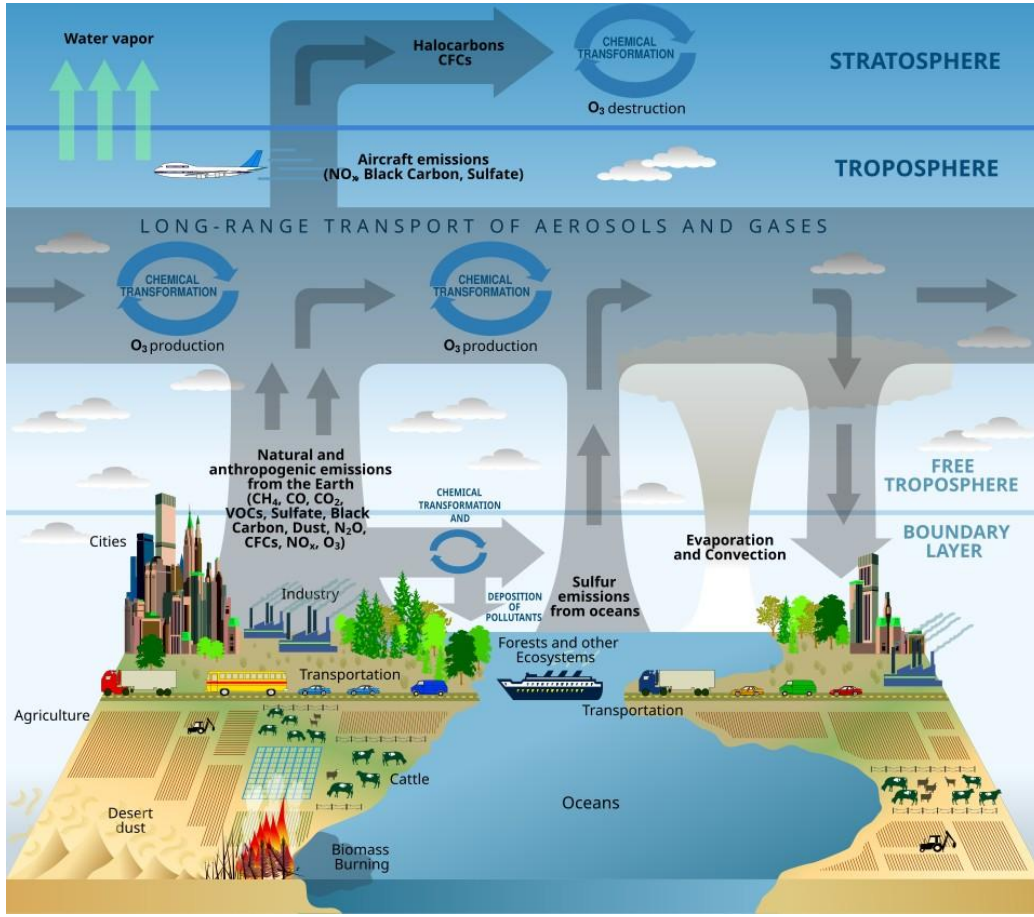
Çoklu Miamarisis ajanlarının bir araya gelerek çalıştığı durumlarda, iletişim ve koordinasyon hayati önem taşır. Şekil 7'deki kamçı taban diyagramı, bir hücrenin karmaşık ve koordineli hareketini sağlayan mikroskobik bir yapıyı tasvir eder. Bu, çoklu ajan sistemlerindeki iletişim protokollerinin ve koordinasyon mekanizmalarının ne kadar hassas ve entegre olması gerektiğini metaforik olarak açıklıyor.



Şekil 7: Kamçı Taban Diyagramı — Şekil 7: Kamçı Taban Diyagramı — Görsel Kaynağı: Wikimedia Commons

Şekil 7'deki gibi, her bir ajan, kendi hedeflerine ulaşırken aynı zamanda sistemin genel hedeflerine katkıda bulunmak için diğer ajanlarla senkronize bir şekilde hareket etmelidir. Bu, karmaşık problem çözme (örneğin, N-Kraliçe problemi gibi dağıtık problem çözme senaryoları) veya sanal odak grupları gibi senaryolarda kritik öneme sahiptir.

Microsoft'un çoklu ajan referans mimarisi (Şekil 8), bu koordinasyonun nasıl yapılandırılabilirliğine dair somut bir örnek sunuyor.



Şekil 8: Çoklu Ajan Referans Mimarisi — Şekil 8: Çoklu Ajan Referans Mimarisi — Görsel Kaynağı: microsoft.github.io

Şekil 8'de gösterilen bu referans mimarisi, farklı ajan türlerinin (örneğin, kullanıcı arayüzü ajanları, iş mantığı ajanları, veri ajanları) bir mesajlaşma katmanı üzerinden nasıl etkileşim kurduğunu ve ortak bir bilgi tabanını nasıl paylaştığını detaylandırıyor. Bu tür mimariler, büyük ölçekli kurumsal uygulamalarda Miamarisis ajanlarının entegrasyonu için bir yol haritası sunar.

Adım Adım Bir Miamarisis Ajanı İnşa Etmek (Basit Bir Örnek)

Gelin, Python kullanarak çok basit bir LLM destekli Miamarisis ajanının temel döngüsünü oluşturalım. Bu örnek, `algıla`, `düşün` ve `eyle` adımlarını gösteriyor.

```
```python
```

```
import openai # Varsayılan LLM sağlayıcısı olarak OpenAI kullanıyoruz
```

```
import json
```

```
class MiamarisisAgent:
```

```
def __init__(self, name, tools=None):
```

```
 self.name = name
```

```
 self.memory = [] # Geçmiş etkileşimler
```

```

self.tools = tools if tools else {} # Ajanın kullanabileceği araçlar
print(f"{self.name} ajanı başlatıldı.")
def perceive(self, environment_data):
 """Çevreden bilgi algılar."""
 print(f"[{self.name}] Çevreden algılanan: {environment_data}")
 self.memory.append({"role": "environment", "content": environment_data})
 return environment_data
def think(self, current_task):
 """Algılanan bilgiye ve göreve göre düşünür ve bir eylem planı oluşturur."""
 prompt = f"""Sen bir Miamarisis ajanısın. Adın {self.name}.
 Görevin: {current_task}
 Geçmiş etkileşimler: {json.dumps(self.memory[-3:])} # Son 3 etkileşimi hatırlıyoruz
 Kullanabileceğin araçlar: {list(self.tools.keys())}
 Bir sonraki adımın ne olmalı? Bir eylem planı oluştur ve eğer bir araç kullanacaksan,
 'TOOL_CALL: {{"tool_name": "...", "args": {{"key": "value"}}}}' formatında belirt.
 Aksi takdirde, bir sonraki mantıksal adımını veya cevabını belirt.
 """
 try:
 response = openai.chat.completions.create(
 model="gpt-4o-mini", # Veya başka bir uygun LLM
 messages=[{"role": "user", "content": prompt}],
 max_tokens=150
)
 thought = response.choices[0].message.content.strip()
 print(f"[{self.name}] Düşünce: {thought}")
 self.memory.append({"role": "agent_thought", "content": thought})
 return thought
 except Exception as e:
 print(f"[{self.name}] Düşünme hatası: {e}")
 return "Bir hata oluştu, düşünecek bir şey bulamadım."
def act(self, thought):

```

```

"""Düşünceye göre eylem gerçekleştir."""
if thought.startswith("TOOL_CALL:"):
 try:
 tool_call_str = thought[len("TOOL_CALL:").:].strip()
 tool_call = json.loads(tool_call_str)
 tool_name = tool_call["tool_name"]
 tool_args = tool_call["args"]
 if tool_name in self.tools:
 print(f"[{self.name}] Aracı kullanıyor: {tool_name} ile {tool_args}")
 result = self.tools[tool_name](**tool_args)
 self.memory.append({"role": "tool_output", "content": f"{tool_name} sonucu: {result}"})
 return result
 else:
 print(f"[{self.name}] Bilinmeyen araç: {tool_name}")
 return f"Bilinmeyen araç: {tool_name}"
 except json.JSONDecodeError:
 print(f"[{self.name}] Araç çağrısı formatı hatalı.")
 return "Araç çağrısı formatı hatalı."
 except Exception as e:
 print(f"[{self.name}] Araç kullanımı hatası: {e}")
 return f"Araç kullanımı hatası: {e}"
 else:
 print(f"[{self.name}] Eylem: {thought}")
 self.memory.append({"role": "agent_action", "content": thought})
 return thought

```

## Örnek araç fonksiyonları

```

def search_web(query):
 return f"Web'de '{query}' arandı ve ilgili sonuçlar bulundu."

def send_email(to, subject, body):

```

```
return f"E-posta gönderildi: Kime: {to}, Konu: {subject}, İçerik: {body}"
```

## Ajanı başlatma ve çalıştırma

```
if __name__ == "__main__":
```

## OpenAI API anahtarınızı ayarlayın

```
openai.api_key = "YOUR_OPENAI_API_KEY"
```

```
my_tools = {
 "web_search": search_web,
 "email_sender": send_email
}

task_agent = MiamarisisAgent("Görev Yöneticisi", tools=my_tools)
```

## Senaryo 1: Basit bir görev

```
task_agent.perceive("Hava durumu bugün nasıl?")

thought = task_agent.think("Bugünkü hava durumunu öğren ve bana bildir.")

task_agent.act(thought)

print("\n" + "="*50 + "\n")
```

## Senaryo 2: Araç kullanımı gerektiren bir görev

```
task_agent.perceive("Acil bir e-posta göndermem gerekiyor.")

thought = task_agent.think("Önemli bir e-postayı 'destek@example.com' adresine 'Acil Destek Talebi' konusuyla gönder.")

task_agent.act(thought)
...
```

Bu kod bloğu, bir Miamarisis ajanının temel `perceive`, `think` ve `act` döngüsünü gösterir. `think` aşamasında LLM, ajanın geçmişini, görevini ve mevcut araçlarını kullanarak bir sonraki eylemi veya düşüncüyü belirler. Eğer bir araç kullanması gerekiyorsa, bunu özel bir formatla belirtir ve `act` aşamasında bu araç çağrılır.

**PRO TIP / UZMAN TAVSİYESİ:**

**Pro Tip:** Ajanınızın "hafızasını" yönetmek çok önemlidir. LLM'lerin bağlam penceresi sınırlı olduğundan, tüm geçmiş etkileşimleri LLM'ye göndermek yerine, özetleme teknikleri (summarization) veya akıllı bellek retrieval (örneğin, en alakalı geçmiş etkileşimleri çekme) kullanın. Bu, hem maliyeti düşürür hem de ajanın daha verimli düşünmesini sağlar.

**NOT ALINIZ:**

❑ **Common Pitfall:** Ajanlar için araç (tool) tanımlarken, fonksiyon isimlerinin ve parametrelerinin LLM tarafından kolayca anlaşılabilir ve doğal dille eşleşir olmasına dikkat edin. Belirsiz veya çok teknik isimler, LLM'nin aracı doğru zamanda ve doğru argümanlarla kullanmasını zorlaştırabilir. Ayrıca, her aracın açık bir şekilde ne yaptığını ve hangi girdileri beklediğini LLM'ye prompt içinde açıklayın.

**Karşılaştırmalı Analiz: Geleneksel Ajanlar vs. Miamarisis Ajanları**

Hiçbir mimari kusursuz değildir ve her yaklaşımın kendi mühendislik tavizleri vardır. Miamarisis mimarisinin geleneksel ajan yaklaşımlarına göre avantajlarını ve dezavantajlarını bir tabloda inceleyelim:

Özellik / Yaklaşım	Geleneksel Kural Tabanlı Ajanlar	Geleneksel Sembolik Ajanlar (SOAR vb.)	Modern LLM Destekli Miamarisis Ajanları
<b>**Esneklik / Adaptasyon**</b>	Düşük. Sadece tanımlı kurallara uyar. Yeni durumlara adapte olamaz.	Orta. Dünya modeli ve planlama ile bir dereceye kadar adapte olur.	Yüksek. Doğal dil anlama, öğrenme ve RAG ile dinamik adaptasyon.
<b>**Karmaşık Akıl Yürütme**</b>	Düşük. Basit mantıksal çıkarımlar.	Orta. Mantık tabanlı çıkarım ve planlama.	Yüksek. LLM'lerin geniş bilgi ve akıl yürütme yetenekleri.
<b>**Geliştirme Hızı**</b>	Hızlı (basit görevler için).	Yavaş. Bilgi mühendisliği ve kural setleri karmaşık.	Orta-Hızlı. LLM'leri entegre etmek kolay, prompt mühendisliği zaman alır.
<b>**Ölçeklenebilirlik**</b>	Düşük. Kural sayısı arttıkça karmaşıklık patlar.	Orta. Bilgi tabanı büyüdükçe performans düşer.	Yüksek. Modüler yapı ve bulut tabanlı LLM'ler ile ölçeklenebilir.
<b>**Açıklanabilirlik**</b>	Yüksek. Kararlar kurallara göre izlenebilir.	Yüksek. Planlama adımları ve mantık izlenebilir.	Orta-Düşük. LLM'lerin "kara kutu" doğası nedeniyle zor.
<b>**Maliyet**</b>	Düşük (basit sistemler için).	Orta-Yüksek. Uzman mühendislik gerektirir.	Yüksek (LLM API maliyetleri ve hesaplama gücü).

<b>**Kullanım Alanı**</b>	Basit otomasyon, kontrol sistemleri.	Uzman sistemler, tanı sistemleri.	Sanal asistanlar, otonom yazılım mühendisliği, araştırma.
---------------------------	--------------------------------------	-----------------------------------	-----------------------------------------------------------

---

**NOT ALINIZ:**

❑ **KRİTİK DARBOĞAZ / MÜHENDİSLİK SINIRI:** Modern LLM destekli Miamarisis ajanlarının en büyük darboğazlarından biri, **hesaplama maliyeti ve gecikme süresidir**. Her düşünce döngüsünde bir LLM çağrısı yapmak, özellikle gerçek zamanlı veya yüksek frekanslı etkileşim gerektiren sistemlerde ciddi performans sorunlarına yol açabilir. Bu, Small Language Models (SLM'ler) veya yerel olarak çalışan optimize edilmiş modellerin önemini artırıyor. Enerji verimliliği de bu noktada kritik bir mühendislik kısıtı olarak karşımıza çıkıyor.

---

**Kapanış: Miamarisis ile Geleceğe Bakış**

Miamarisis ajan mimarisi, yapay zeka alanında karşılaştığımız en zorlu problemleri çözmek için bize güçlü bir çerçeve sunuyor. Otonom araçlardan akıllı şehir sistemlerine, kişiselleştirilmiş eğitim platformlarından sanal araştırma asistanlarına kadar birçok alanda devrim yaratma potansiyeline sahip. Özellikle, **SWEnergy** gibi çalışmaların gösterdiği enerji verimliliği odaklı yaklaşımlar ve **Focus Agent** gibi LLM destekli sanal odak grupları, bu ajanların gelecekteki uygulama alanlarının genişliğini gözler önüne seriyor.

Ancak unutmayalım ki bu teknolojiler hala gelişim aşamasında. Etik kullanım, şeffaflık, güvenlik ve insan-ajan işbirliği gibi konular, üzerinde çalışmaya devam etmemiz gereken kritik alanlar.

Peki siz ne düşünüyorsunuz? Miamarisis ajanlarının gelecekte hayatımızda ne gibi roller oynayacağını hayal ediyorsunuz? Yorumlarda buluşalım ve bu heyecan verici geleceği birlikte tartışalım!

---



## KAYNAKÇA VE BİBLİYOGRAFYA

- [1] Zhu, C., Dastani, M., Wang, S. (2022). "A Survey of Multi-Agent Deep Reinforcement Learning with Communication". arXiv Preprint. [ Erişim: <http://arxiv.org/abs/2203.08975v2>](<http://arxiv.org/abs/2203.08975v2>)
- [2] Zhang, T., Zhang, X., Cools, R. (2024). "Focus Agent: LLM-Powered Virtual Focus Group". arXiv Preprint. [ Erişim: <http://arxiv.org/abs/2409.01907v1>](<http://arxiv.org/abs/2409.01907v1>)
- [3] Tripathy, A., Harshit, C. P., Vaidhyanathan, K. (2025). "SWEnergy: An Empirical Study on Energy Efficiency in Agentic Issue Resolution Frameworks with SLMs". arXiv Preprint. [ Erişim: <http://arxiv.org/abs/2512.09543v2>](<http://arxiv.org/abs/2512.09543v2>)
- [4] Han, J., Liu, J., Cai, Q. (2002). "From Alife Agents to a Kingdom of N Queens". arXiv Preprint. [ Erişim: <http://arxiv.org/abs/cs/0205016v1>](<http://arxiv.org/abs/cs/0205016v1>)
- [5] Takayama, S., Fujita, K. (2025). "AOAD-MAT: Transformer-based multi-agent deep reinforcement learning model considering agents' order of action decisions". arXiv Preprint. [ Erişim: <http://arxiv.org/abs/2510.13343v1>](<http://arxiv.org/abs/2510.13343v1>)
- [6] Kiourt, C., Kalles, D. (2018). "How game complexity affects the playing behavior of synthetic agents". arXiv Preprint. [ Erişim: <http://arxiv.org/abs/1807.02648v1>](<http://arxiv.org/abs/1807.02648v1>)
- [7] Arı Bilgi Eğitim Kurumları. "LLM Agent Mimarisi". [ Erişim: <https://aribilgi.com/llm-agent-mimarisi>](<https://aribilgi.com/llm-agent-mimarisi>)
- [8] Gupta, A. (2023). "The Complete Guide to AI Agent Architecture". Medium (Towards Data Science). [ Erişim: <https://medium.com/@amarg3891/the-complete-guide-to-ai-agent-architecture-25dc2cbe7016>](<https://medium.com/@amarg3891/the-complete-guide-to-ai-agent-architecture-25dc2cbe7016>)
- [9] Microsoft. "Multi-Agent Reference Architecture". [ Erişim: <https://microsoft.github.io/multi-agent-reference-architecture/docs/reference-architecture/architecture.drawio.svg>](<https://microsoft.github.io/multi-agent-reference-architecture/docs/reference-architecture/architecture.drawio.svg>)
- [10] ResearchGate. "Agent Architecture: An Overview". [ Erişim: [https://www.researchgate.net/publication/275643980\\_Agent\\_Architecture\\_An\\_Overview](https://www.researchgate.net/publication/275643980_Agent_Architecture_An_Overview)]([https://www.researchgate.net/publication/275643980\\_Agent\\_Architecture\\_An\\_Overview](https://www.researchgate.net/publication/275643980_Agent_Architecture_An_Overview))
- [11] TOPALOĞLU, F. (2021). "Sis Hesaplama da Sis Döğümlerinin Rolü ve Mimari Yapısı". European Journal of Science and Technology. [ DOI: <https://doi.org/10.31590/ejosat.1018653>](<https://doi.org/10.31590/ejosat.1018653>)

---